MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión E del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_bravo_paulo
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/bravo_paulo/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué significa que un objeto sea una instancia de una clase? Explica si dos instancias deben tener iguales valores en sus atributos.

Tu respuesta

```
significa que ese objeto fue creado en el mismo plano (que es la clase)
```

Corrección. Reconoces que el objeto se crea a partir de la clase, que funciona como plano. Falta la otra mitad de la idea: dos instancias comparten la definición de sus atributos y métodos, pero cada una guarda sus propios valores de instancia, que pueden ser distintos.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un parámetro y un argumento de un método? Ilústrala con una llamada sencilla.

Tu respuesta

```
la diferencia es donde se define cada una y donde se radica
```

Corrección. Solo apunta de forma vaga a que cada uno se define en un lugar distinto, sin decir cuál es cuál y sin ejemplo. El parámetro es la variable declarada en la cabecera, como int cantidad en cargar(int cantidad); el argumento es el valor que se pasa en la llamada, como el 3 en cargar(3).

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué es el ámbito de una variable local? Da un ejemplo de dónde se puede usar y dónde deja de estar disponible.

Tu respuesta

```
la variable local es la q funciona solamente en ese codigo es decir no es una global para q pueda ser usada en varios codigos, deja de estar disponible cuando el codigo termina
```

Corrección. Capta que la variable local solo se puede usar en una parte del código y deja de existir al salir de ella. Queda impreciso ("ese código") y falta un ejemplo concreto: una variable declarada dentro de las llaves de un if o de un método se puede usar dentro de ese bloque, no fuera de él.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo arreglo? Explica qué ocurre al asignarla a otra variable del mismo tipo.

Tu respuesta

```
una variable de tipo arreglo puede almacenar varios datos de numeros o de nombres etc, si se le asigna a otra esa nueva va a tener los valores de el primer arreglo
```

Corrección. Es cierto que un arreglo agrupa varios datos, pero la variable no guarda los datos: guarda una referencia al arreglo. Al asignarla a otra variable se copia esa referencia y ambas apuntan al mismo arreglo (son alias), así que un cambio hecho por una se ve en la otra. La respuesta sugiere una copia de valores.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace break dentro de un case de un switch clásico? Explica qué puede ocurrir si falta.

Tu respuesta

```
el brake va al final de cada caso dentro de un switch, si en algun caso falta el break la computadora se seguira al siguiente caso.
```

Corrección. Bien: si falta break, la ejecución sigue con el case siguiente. Faltó decir de forma explícita que break sale del switch y continúa con la instrucción que viene después.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer original = 7;
Integer copia = original;
int vuelta = 0;
do {
    original = original + 2;
```

```
    System.out.println(original / 2);
    vuelta++;
} while (vuelta < 3);
String texto = "E" + copia + 1;
System.out.println(texto);
System.out.println((double) (original / 2));
```

Tu respuesta

```
1er print- va a imprimir 4 y ademas le suma 1 a la vuelta
2do print- se va a imprimir "E" + 10
3er print- va a imprimir 4.5 por el double
```

Corrección. La primera línea, 4, es correcta. El do-while se ejecuta tres veces y también imprime 5 y 6. Luego viene E71, porque "E" + copia + 1 concatena de izquierda a derecha; la E inicial sí es correcta. La última línea es 6.0, porque 13/2 es una división entera y después se convierte a double; 4.5 es incorrecto.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = null;
boolean igual = codigo.equals("R3");
System.out.println(igual);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
porque esta diciendo que codigo es =null
```

Corrección. La clasificación es incorrecta: el código sí compila, pero truena en ejecución. La causa apunta bien al null, pero faltó lo concreto: se invoca equals sobre una referencia null, lo que lanza NullPointerException antes del println.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double voltaje = 7.6;
int lectura = voltaje;
System.out.println(lectura);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
va a imprimir 7 porque esta diciendo que lectura es un entero entonces solo toma la parte sin decimales
```

Corrección. Es incorrecta: no compila, porque Java no convierte double a int de forma implícita (se pierde precisión). Haría falta un cast explícito, (int) voltaje, y solo entonces imprimiría 7.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer reserva = 4;
int nueva = reserva + 3;
System.out.println(nueva);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
por el integer no se puede sumar con un int
```

Corrección. Es incorrecta: funciona e imprime 7. Java desempaqueta el Integer automáticamente (unboxing) para sumarlo con el int.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada banda nace vacía con capacidad propia fija. El límite de seguridad común es 12 piezas. agregarPieza() suma una y consultarPiezas() responde cuántas hay. Supón que solo se llama agregarPieza cuando ambos límites lo permiten.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cuál es final de instancia y cuál static final: capacidad o límite de seguridad?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
Class Banda;
static limiteSeguridad = 12;
int agregarPieza = 1;
consultarPiezas = 0;
int piezas = 0;
static capacidadFija = 0;

if (piezas < 12 && capacidadFija <= 0);{
    consultarPiezas = consultarPiezas + agregarPieza;
    Systemout.println(consultarPiezas);
    }
```

Justificaciones

```
se usa el ciclo if para saber si las piezas son menores al limite de seguridad, posteriormente se usa consultar piezas y se le suma 1 mientras se cumplan las condiciones y al final se imprimen cuantas piezas se tienen.
```

Corrección. Solo piezas está bien declarada como int de instancia, y falta justificarla. Además: capacidad se hizo static cuando debe ser final de instancia; el límite no tiene tipo ni es final; agregarPieza y consultarPiezas se declararon como variables en lugar de métodos; no hay constructor; se usó un if, que el enunciado excluye; y no se responde el reto. Modelo esperado: final int capacidad; int piezas; static final int LIMITE_SEGURIDAD = 12; un constructor con this.capacidad = capacidad; agregarPieza() { piezas++; } y consultarPiezas() { return piezas; }.

R4 · Código con repetición

Pregunta

Un equipo parte con carga unidades. En cada paso suma aporte. Devuelve cuántos pasos necesita para alcanzar o superar meta usando while. Si ya llegó, devuelve 0. Carga y meta están entre 0 y 10000; aporte está entre 1 y 100. No uses una fórmula de división.

```
static int pasosCarga(int carga, int aporte, int meta)
```

Ejemplos (argumentos → retorno):

5, 4, 16 → 3 · 16, 4, 16 → 0 · 20, 4, 16 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int pasosCarga(int carga, int aporte, int meta)
```

```
int meta = [0,10000];
int aporte = [1,100];
int carga = [0,10000];
int pasosFaltantes = 0;
int pasos = 0;
pasos = carga + aporte;

while (pasos < meta);{
pasosFaltantes = meta - pasos
Systemout.println("NECESITAS" + pasosFaltantes + "PASOS PARA LLEGAR A LA META");
}

while (pasos >= meta);
{
return 0;
}

// LO Q ENTENDI FUE EL EQUPO AVANZA CON carga UNIDADES Y EN CADA PASO SUMA aporte, despues va
a imprimir el numero de pasos q le hacen falta usando un while para saber si ya llego o no a
la meta, si ya llego a la meta solo retorna 0, le asigne esos valores porque creo q asi se
ponian intervalos de numeros.
```

Corrección. La cabecera es correcta y usa while comparando contra meta. Van bien sumar aporte a carga una vez, la intención de devolver 0 si ya llegó y la condición de paro. La suma ocurre fuera del ciclo, no se cuentan pasos, el ';' tras while y el cuerpo sin avance hacen que el ciclo no termine, se redeclaran los parámetros, se imprime y falta el return del caso general. Lo esperado: int pasos = 0; while (carga < meta) { carga += aporte; pasos++; } return pasos;

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Decide si un banco de pruebas puede arrancar. Recibes el porcentaje de batería y si la tapa está cerrada. Aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else, con && y ||. No imprimas ni leas del teclado.

```
static String permiso(int bateria, boolean tapaCerrada)
```

Regla o valor recibido	Texto devuelto
Batería menor que 0 o mayor que 100	"INVALIDA"
Tapa cerrada y batería al menos 60	"NORMAL"
Tapa cerrada y batería al menos 20	"AHORRO"
Cualquier otro caso	"BLOQUEADO"

Tu respuesta

```
static String permiso(int bateria, boolean tapaCerrada)

if (tapaCerrada == true && bateria < 20);{
    Systemout.println("AHORRO");
```

```
   }
else
  if (tapaCerrada == true && bateria >= 60);{
      Systemout.println("NORMAL");
      }
   else
      if (bateria < 0, bateria > 100);{
          Systemout.println("INVALIDA")
          }
        else
          System.println("BLOQUEADO")
```

Corrección. La cabecera es correcta y la cadena if/else-if/else está bien armada, aunque usa una coma en lugar de ||. La conjunción de NORMAL es correcta y el else BLOQUEADO también; la tapa abierta con batería alta sí queda bloqueada. Errores: AHORRO usa bateria < 20 en lugar de bateria >= 20 (excluye el 20), INVALIDA se revisa al final cuando debía ser la primera regla (así -1 con tapa cerrada da AHORRO), se imprime en lugar de usar return y hay ';' tras cada if.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.